SECTION 2 CHEAT SHEET

Read the answer from one forward pass

A decision model never writes its answer: each option's probability comes from the logits. **Expected calibration error** checks those numbers and **temperature scaling** adjusts them.

Ticket 2: the whole team is locked out

gold answer technical

QUESTIONS Choice Score Noul

tev1	account	0.52
nimble	technical	0.84
clef-flash	technical	0.95

probability of each model's pick

1 ONE REQUEST

A state and typed questions in, probabilities out

One `system_one` call answers every typed question: **Choice** picks an option, **Score** a level, **Noul** yes or no. Only `model` changes between the Ollama models.

```
client = TypeSafeClient(api_key="ollama",
    base_url="http://127.0.0.1:11434", timeout=300)
QUESTIONS = {"department": Choice(...),
             "urgency": Score(...), "refund": Noul(...)}
response = client.system_one(ticket, QUESTIONS,
                             model=model_name)
```

COURSE RUN tev1 on ticket 0: billing **0.998**, urgency **1.10**, refund **0.95**.

Choice **department** 4 options
Which team should handle this ticket?

Score **urgency** 3 levels
How urgent is this ticket?

Noul **refund** yes or no
The customer explicitly asks for money back.

Score levels: can wait, this week, today

2 TYPED QUESTIONS

Choice, Score, Noul

A question is its type, an instruction and criteria: **Choice** names options, **Score** lists levels from low to high, **Noul** needs only the instruction.

```
QUESTIONS = {
    "department": Choice(
        criteria={"billing": ..., "sales": ...}),
    "urgency": Score(
        criteria=["can wait", "this week", "today"]),
    "refund": Noul(
        instructions="The customer explicitly asks ...")
}
```

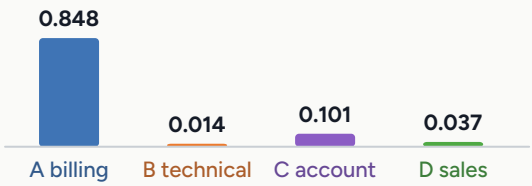
COURSE RUN `Noul` prints **criteria=None**: the instruction is the whole question.

prompt: A billing B technical C account D sales

Qwen / **Qwen2.5-0.5B-Instruct**



the last position: 151,936 scores, keep four



3 ONE FORWARD PASS

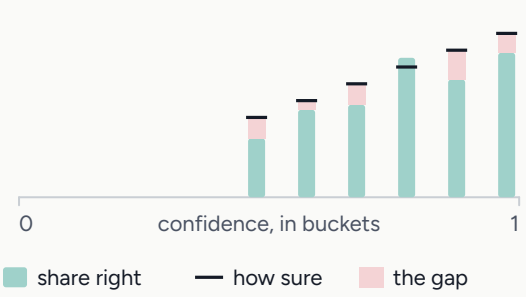
Read the logits, never ask for JSON

Options behind letters, one forward pass, then softmax over the four letter logits at the last position. The answer can only be one of your options.

```
for letter in LETTERS:
    LETTER_IDS.append(
        tok.encode(letter, add_special_tokens=False)[0])
with torch.no_grad():
    logits = model(**x).logits
last = logits[0, -1]
letter_logits = last[LETTER_IDS].float().cpu()
probs = torch.softmax(letter_logits, dim=-1)
```

GENERATE JSON OR READ THE LOGITS: THE SAME QWEN, THE SAME TICKETS

	GENERATE JSON	READ THE LOGITS
Forward passes	one per output token	one
Output	text to parse; **3 of 8** were JSON	a probability per option
Confidence	a number it wrote (0.95 on ticket 0)	measured from the logits
Time per ticket (run)	**1,439 ms**	**46.6 ms**



4 SURE AGAINST RIGHT

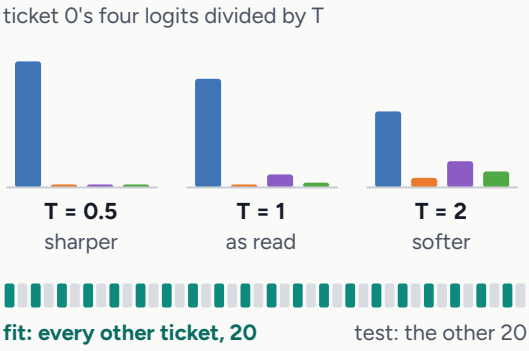
Accuracy, then expected calibration error

The top probability is the confidence, its option the pick. ECE weighs the gap in each confidence bucket: $\Sigma_b\ (n_b/N)\ |acc_b - conf_b|$, zero is perfect.

```
probs = torch.softmax(L, dim=-1)
conf, pred = probs.max(dim=-1)
correct = pred == gold
correct.float().mean()

from laya.evals import ece
ece(conf.tolist(), correct.tolist())
```

COURSE RUN **28 of 40** right (in words, Part 1: **16 of 40**); ECE **0.115**.



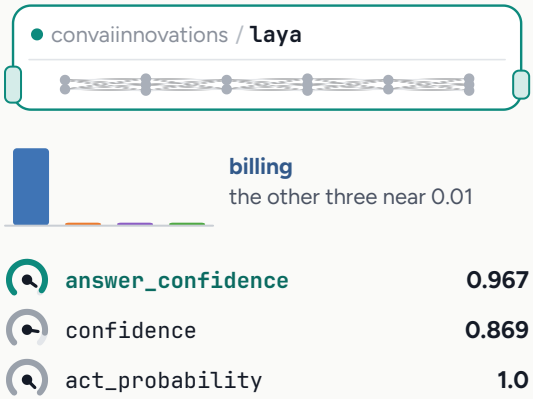
5 TEMPERATURE SCALING

Divide the logits by one number

softmax(logits / T): above 1 softer, below 1 sharper, and the order never changes, so accuracy stays the same. Fit T by the lowest log loss on labelled rows, never on the test rows.

```
fit = slice(0, 40, 2)
for T in [0.5, 0.6, 0.7, 0.8, 0.9, 1, 1.25, 1.5, 2, 3, 4]:
    loss = torch.nn.functional.cross_entropy(
        L[fit] / T, gold[fit])
    losses[T] = loss.item()
best_T = min(losses, key=losses.get)
```

COURSE RUN lowest log loss at T **0.9** (**0.546**, against **0.549** at T 1). On the test half ECE went from **0.285** to **0.338**: worse.



6 LAYA'S THREE NUMBERS

Trust answer_confidence

`answer_confidence`: the probability of the answer, the calibrated one ECE and Brier read. `confidence`: how spread out all the options are. `act_probability`: a separate act or escalate head.

```
agent = laya.Agent("convaiinnovations/laya", device="cuda")
result = agent.predict(tickets["text"][0], LAYA_QUESTIONS)
answer = result["answers"]["department"]
answer["answer_confidence"]
```

COURSE RUN Laya reads with **ModernBERT-large**.

runs/tickets.jsonl, one ticket per line

```
"state":
"I was charged twice for my ..."
"questions":          the three questions
"expected":
   department          billing
   urgency               1
   refund              true
"tags":              ["billing"]
```

7 LABELLED ROWS

One JSON line per ticket, one report

Each row holds the `state`, the `questions`, the `expected` answers and `tags` to slice by. `evaluate` runs every row through the agent and scores it.

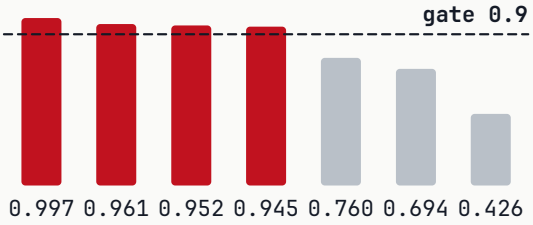
```
rows.append({
    "state": ticket["text"],
    "questions": LAYA_QUESTIONS,
    "expected": {"department": ticket["department"], ...},
    "tags": [ticket["department"]]
})
table.to_json("runs/tickets.jsonl", orient="records",
              lines=True)
data = Dataset.from_jsonl("runs/tickets.jsonl")
report = evaluate(agent, data)
```

COURSE RUN **40** rows; department **35 of 40**, refund **0.95**, urgency off by **0.86** levels.

department right, by ticket tag



Laya's 7 wrong answers, surest first



8 SLICE, THEN READ THE MISTAKES

Where Laya is weak

`report.slices` splits every metric by question or by tag; `report.cases` holds one row per answer, so the wrong ones can be sorted surest first.

```
by_question = pd.DataFrame.from_dict(
    report.slices["qid"], orient="index")
by_department = pd.DataFrame.from_dict(
    report.slices["tag"], orient="index")
cases = pd.json_normalize(report.cases)
wrong = cases[cases["correct"] == False]
wrong = wrong.sort_values("confidence", ascending=False)
```

COURSE RUN ECE: department **0.348**, refund **0.079**. Technical was the weak tag at **0.750**; the surest mistake, **0.9973**, called a sales ticket billing.

TERMS

option letters: the A to D the answer is read from.

probability, confidence, score: one option's number; the picked option's probability; a Score question's value.

expected calibration error (ECE): how far confidence sits from accuracy, in one number.

temperature scaling: logits divided by one fitted number, not the sampling temperature.

WHAT EACH CALL HANDS BACK

`model(**x).logits`
[1, 103, 151936] text, tokens, vocabulary

`torch.stack(rows)`
[40, 4] one row per ticket

`ece(confidences, corrects)`
one number, lower is better

`agent.predict(state, questions)`
a dict, one answer per question

`evaluate(agent, data)`
a report: .overall, .slices, .cases

WATCH OUT

- A confidence gate lets sure mistakes through: **4** of Laya's **7** wrong answers sat above 0.9.
- A temperature fitted on 20 tickets made the other 20 worse: ECE **0.285** to **0.338**.
- Gate on `answer_confidence`, not on the field called `confidence`.